Learning Agentic AI: The 20% That Matters

1 What "Agentic" Actually Means

An agent is an LLM placed inside a **loop** where it can decide its own next step and **act on the world through tools**, rather than producing one fixed response. The defining property is **dynamic control flow**: the path through the program is decided at runtime by the model, not hard-coded by you.

Think of a spectrum:

- **Single call** — one prompt, one answer. Not an agent.
- **Workflow** — a fixed sequence of LLM calls you wired up. Predictable; not really agentic.
- **Agent** — the model chooses which tools to call and when to stop. Adaptive; this is agentic AI.

More autonomy means more capability *and* more unpredictability. The whole discipline is about getting the capability without the chaos.

2 The Thesis (Read This Twice)

Most production agent failures between 2024 and 2026 were **architectural, not model-quality** failures. The LLM worked fine; the design around it didn't. The canonical horror story: a system halts in an infinite loop, burns through hundreds of dollars of API credits, and returns nothing — and the model was the last thing that needed fixing. Consequently, **designing agent control flow is now the highest-leverage skill in AI engineering**. This document is about that design.

3 The Agent Loop (ReAct)

The foundational, most battle-tested pattern is **ReAct** (Reasoning + Acting). The agent alternates between thinking and doing:

```
Thought  -> what should I do next?
Action   -> call a tool (with arguments)
Observation -> read the tool's result
Thought  -> what did I learn? what next?
... repeat until ...
Final answer
```

Why it works: ReAct handles tasks where you don't know the full path upfront. If a tool call fails, the agent tries another approach; if a search returns surprising data, it adjusts. That adaptivity is what makes agents useful for dynamic tasks instead of scripted ones. **Default to ReAct** and only add complexity when it hits a ceiling.

4 Tools / Function Calling (The Core Capability)

Tools are how an agent *does* things: search, query a database, call an API, run code, send an email. Modern models support **structured tool calling with schema validation** — the model returns a structured request (tool name + typed arguments) that your code executes. The state of the art is consistent across OpenAI, Anthropic, and capable open-source models.

```
# A tool is a function + a clear description + a typed signature
def search_orders(customer_id: str, status: str = "all") -> list:
    """Look up a customer's orders. Use when the user asks about an order.
       """
    ...
```

Tool design is most of agent quality. Treat tools like a strict API contract:

- A precise **description** (the model decides whether to call it by reading this).
- **Typed, validated arguments** so a malformed call is caught, not executed.
- **Narrow scope** — many small clear tools beat one giant ambiguous one.
- **Safe failure** — return a useful error the agent can reason about, don't crash.

5 The Design Patterns (The Architectural Vocabulary)

These are the reusable building blocks. You combine them; no real system uses just one.

- **Tool Use** — the agent calls external functions/APIs. The bedrock capability.
- **ReAct** — interleaved reason/act loop (above). Your default.
- **Reflection** — the agent critiques and revises its own output (an "evaluator" checks a "generator"). Improves quality on fuzzy tasks, but costs extra model calls each cycle — use it where quality clearly justifies the spend, not everywhere.
- **Planning (Plan-and-Execute)** — the agent writes an explicit plan of subtasks first, then executes it. Reduces "cognitive entropy" (agents losing track of the overall goal deep inside subtasks) and is cheaper/more predictable than open-ended ReAct for well-understood, multi-step jobs.
- **Multi-Agent Collaboration** — several specialized agents (e.g. researcher, writer, reviewer) work together.
- **Orchestrator–Worker** — a coordinator agent delegates subtasks to workers and combines results.
- **Human-in-the-Loop** — the agent pauses for human approval at sensitive steps. Essential for anything touching real money or regulated actions.

They layer. A real customer-service agent might be: Tool Use (CRM/order APIs) + ReAct (diagnose the issue) + Human-in-the-Loop (escalate refunds above a threshold) + a sequential workflow for standard paths.

6 Single-Agent First, Multi-Agent Only When Forced

The most common mistake is jumping to multi-agent before a single agent has reached its quality ceiling. Multi-agent adds roughly 2–5× the coordination overhead and far more debugging surface, and the quality gain is often modest unless the task is genuinely decomposable. The discipline:

1. Build a single ReAct agent.
2. Measure success rate, tool-call accuracy, latency, and cost.
3. Identify the specific failure mode you *cannot* fix single-agent.
4. Only then escalate to multi-agent — to solve that named problem.

7 Matching the Pattern to Your Constraint

There is no universally best pattern; match it to your most binding constraint:

- **Need real-time adaptivity, cost is acceptable** → ReAct.
- **Predictable sequence, cost-sensitive** → Plan-and-Execute / sequential workflow.
- **Quality-critical, fuzzy output** → add Reflection.
- **Regulated / high-stakes** → Human-in-the-Loop is mandatory.
- **Genuinely parallel sub-problems** → multi-agent.

8 Memory

Agents need state beyond a single turn:

- **Short-term / working memory** — the running conversation and intermediate results, held in the context window (and checkpointed, as in the LangGraph document).
- **Long-term memory** — persistent, queryable storage of facts, preferences, and past outcomes across sessions (often a vector store, as in RAG). Increasingly practical rather than experimental.

The skill is deciding what to keep, what to summarize, and what to retrieve — context is finite and every token costs money and latency.

9 Standards: MCP and Friends

The ecosystem is converging on interoperable standards so tools and agents are reusable across systems:

- **MCP (Model Context Protocol, Anthropic)** — a standard way to expose tools, data, and resources to any agent. Write an MCP server once; any MCP-aware agent can use it.
- **A2A (Agent2Agent, Google)** — a protocol for agents to talk to each other.
- **Vendor Agents SDKs** (e.g. OpenAI's) — opinionated toolkits for the loop above.

The direction of travel: composable, third-party agent capabilities you assemble rather than rebuild.

10 Failure Modes and Safeguards (Non-Negotiable)

Agents fail in ways single calls don't. Build these guards *before* you let an agent run unattended:

- **Infinite loops** → a hard **max-iteration** cap on the loop.
- **Cost runaway** → a **token/dollar budget** per task that aborts when exceeded.
- **Stuck / hanging tools** → **timeouts** on every tool call.
- **Error cascades** → catch tool errors and feed them back as observations, with a retry limit.
- **Cognitive entropy** (losing the goal) → keep an explicit plan/goal in context; re-state it.
- **Unsafe actions** → human approval gate for irreversible or high-stakes steps.

The $300-loop story above is what happens when the max-iteration cap and budget are missing.

11 Evaluating Agents

Evaluating an agent means judging the **whole trajectory**, not just the final message (as in the GenAI-systems document). Track: was each **tool choice** correct, were the **arguments** valid, how many **steps**, what **time and cost**, and did it follow **policy**? Use deterministic tool **mocks** in CI so tests are repeatable, and add LLM-as-judge scoring only with an explicit rubric and human auditing.

12 Agentic RAG (Where Two Topics Meet)

Plain RAG always retrieves. **Agentic RAG** lets the agent *decide*: whether to retrieve at all, which source to query, whether to reformulate the question, and whether the retrieved context is good enough or it should search again. Retrieval becomes a tool the ReAct loop calls when it judges it useful — more flexible, at the cost of more model calls.

13 When NOT to Use an Agent

Agents add cost, latency, and unpredictability. If a deterministic workflow or a plain script does the job, use that. Reserve agentic autonomy for tasks where the path genuinely can't be known in advance. "Could this be a fixed pipeline?" is a question worth asking before every agent you build.

14 The Frameworks Landscape

You implement these patterns with an orchestration framework. **LangGraph** (covered in its own document) models the agent as an explicit graph and is strong for custom control flow; CrewAI and AutoGen lean toward multi-agent setups; vendor SDKs offer quick paths. The patterns in this document are framework-independent — learn them once, apply them anywhere.

15 The Checklist for Any Agent

1. Ask first: does this even need an agent, or will a workflow do?
2. Start with a single **ReAct** agent and well-designed **tools** (clear descriptions, typed args).
3. Add **safeguards**: max iterations, budget, timeouts, error handling.
4. Add **memory** only as needed (short-term first, long-term if it must persist).
5. Add **human-in-the-loop** for anything irreversible or high-stakes.
6. **Evaluate the trajectory**; measure success, tool accuracy, steps, cost.
7. Escalate to reflection / planning / multi-agent only to fix a *named* failure mode.

16 Practice Task

Build a single ReAct agent and harden it:

1. Give it two tools: a calculator and a (real or fake) search. Write clear descriptions and typed arguments.
2. Run it on a question needing both tools; print the full Thought/Action/Observation trace.
3. Add a **max-iteration cap** and a per-task **token budget**; deliberately trigger each and confirm it stops cleanly.
4. Make a tool fail on purpose; confirm the agent reads the error and recovers instead of crashing.
5. Add a **reflection** step that critiques the final answer once, and measure the extra cost — decide if it was worth it.
6. Write 5 test tasks with expected tool-call sequences; score the agent's *trajectory*, not just its answer.
7. Bonus: turn the search tool into your RAG retriever (agentic RAG), implement it in LangGraph, serve the model with vLLM, expose it via FastAPI, and ship it in Docker.